

Pomysł na standard projektowania struktury repozytoriów

Chcemy dodać do `wellmanifest/new-project` osobny plik:

`REPOSITORY_STRUCTURE.md`

Plik będzie zawierał krótkie zasady, według których AI ma projektować strukturę nowego repozytorium.

Nie chcemy tworzyć jednego sztywnego drzewa katalogów obowiązującego w każdym projekcie, ponieważ projekty mogą używać różnych języków, technologii i sposobów podziału systemu.

AI ma dobrać strukturę na podstawie rzeczywistych wymagań projektu.

Główna zasada

Struktura repozytorium ma być na tyle czytelna, aby po wejściu do projektu można było bez przeglądania kodu zrozumieć:

- z jakich części składa się system;
- które części są osobno uruchamiane;
- gdzie znajduje się dana funkcjonalność;
- gdzie są testy;
- gdzie jest konfiguracja;
- gdzie znajduje się infrastruktura;
- jakie technologie są używane w poszczególnych częściach.

Dobór struktury

Dla prostego projektu składającego się z jednej aplikacji wystarczy klasyczna struktura:

```
src/  
tests/  
project/
```

Jeżeli projekt zawiera kilka niezależnych części, nie powinny być one mieszane w jednym wspólnym `src/`.

Przykład:

```
frontend/  
backend/  
worker/  
project/
```

W większym projekcie części mogą zostać pogrupowane według ich roli:

```
applications/  
services/  
libraries/  
infrastructure/  
project/
```

Takie katalogi mają jednak powstawać tylko wtedy, gdy rzeczywiście wynikają z projektu.

Projekty wielojęzyczne

Jeżeli projekt korzysta z kilku języków programowania, każda niezależna część powinna mieć własny katalog.

Przykład:

```
frontend/  
├─ src/  
├─ tests/  
└─ package.json  
  
backend/  
├─ src/  
├─ tests/  
└─ pyproject.toml  
  
worker/  
├─ src/  
├─ tests/  
└─ Cargo.toml
```

Dzięki temu nie powstaje jeden globalny katalog `src/`, w którym są wymieszane różne języki i niezależne aplikacje.

Technologia nie musi zawsze znajdować się w nazwie katalogu. Nazwa katalogu powinna przede wszystkim opisywać jego odpowiedzialność.

Przykłady dobrych nazw:

```
admin-panel/  
payment-api/  
notification-worker/  
document-converter/
```

Jeżeli kilka podobnych komponentów trudno rozróżnić, technologia może zostać dodana do nazwy:

```
payment-api-python/  
payment-worker-rust/
```

Niezależne komponenty

Każdy niezależnie budowany lub uruchamiany komponent powinien posiadać własne:

- źródła;
- testy;
- zależności;
- konfigurację;
- sposób uruchomienia;
- Dockerfile, jeżeli komponent jest uruchamiany jako osobna usługa.

Przykład:

```
payment-api/  
├─ src/  
├─ tests/  
├─ Dockerfile  
├─ README.md  
└─ plik zależności właściwy dla technologii
```

Czego nie tworzyć automatycznie

AI nie powinno tworzyć pustych katalogów na przyszłość.

Dotyczy to szczególnie katalogów:

```
apps/  
services/  
packages/  
shared/  
common/  
utils/  
docs/
```

Mogą one powstać tylko wtedy, gdy mają konkretną odpowiedzialność wynikającą z projektu.

Przykładowo `shared/` nie powinno być miejscem na przypadkowy kod używany w kilku miejscach. Powinno powstać dopiero wtedy, gdy istnieje rzeczywiście wspólny komponent lub kontrakt.

Czytelność ponad uniwersalnym szablonem

Nie zależy nam na tym, aby każde repozytorium wyglądało identycznie.

Zależy nam na tym, aby każde repozytorium było zaprojektowane według tych samych zasad:

- odpowiedzialności są rozdzielone;
- niezależne komponenty są widoczne;
- technologie nie są wymieszane;
- nazwy katalogów jasno opisują ich zawartość;
- nie istnieją zbędne warstwy i puste katalogi;
- struktura pozwala szybko znaleźć potrzebne elementy.

Refaktoryzacja i automatyzacja

Dobra struktura ma umożliwiać przeprowadzenie wstępnej analizy projektu bez ręcznego przeglądania wszystkich plików.

Na podstawie katalogów oraz plików konfiguracyjnych powinno być możliwe automatyczne ustalenie:

- liczby komponentów;
- używanych technologii;
- lokalizacji testów;
- sposobu budowania i uruchamiania;
- potencjalnych zależności między częściami;
- zakresu komponentów dotkniętych refaktoryzacją.

Struktura nie zastępuje analizy kodu, ale znacząco ogranicza ilość kodu, który trzeba ręcznie przeglądać.

Rola katalogu `project/`

Katalog `project/` pozostaje elementem każdego projektu.

Służy do organizowania zmian, ticketów, pracy uczestników i logów.

Jego dokładna nowa struktura zostanie ustalona osobno i nie jest częścią tego pomysłu.

Organizacja zasad

Zasady dotyczące struktury nie powinny być rozbudowywane bezpośrednio w `CONTRIBUTING.md`.

`CONTRIBUTING.md` powinien zawierać jedynie krótkie wskazanie, że przy tworzeniu lub przebudowie repozytorium należy przeczytać:

`REPOSITORY_STRUCTURE.md`

Dzięki temu:

- `CONTRIBUTING.md` pozostanie krótki;
- zasady procesu pracy nie będą wymieszane z zasadami architektury;
- zasady struktury będzie można rozwijać niezależnie;
- AI będzie wiedziało, gdzie znaleźć konkretne instrukcje dotyczące projektowania repozytorium.